# FinOps Design-Time Optimizer

## Baseline Architecture and Design

Version 0.1 | October 2026

**WORKING BASELINE**

This document captures the initial design hypothesis. Components, integration paths, and scope are expected to evolve during the hack.

# 1. Executive summary

The FinOps Design-Time Optimizer is a proposed developer workflow that allows an MCP-enabled coding agent to use cost, pricing, utilization, and optimization context when reviewing application code and infrastructure-as-code. The agent can recommend or implement a candidate optimization, invoke a repeatable load test, and report the estimated cost and performance impact before the change is merged.

The hack will validate the pattern through one controlled workload and one end-to-end optimization scenario. The design intentionally separates the stable product hypothesis from implementation choices that are likely to change during discovery.

# 2. Problem statement

- Cloud-cost feedback commonly arrives after deployment, when architectural choices are more expensive to change.
- Coding agents can refactor applications but usually lack organization-specific cost, usage, utilization, and FinOps context.
- Static price estimates do not show whether a cheaper design still meets throughput, latency, reliability, and error-rate objectives.
- Engineering teams need an evidence trail that connects a recommendation to a tested change and a measurable unit-cost comparison.

# 3. Goals and non-goals

| Goals | Non-goals |
|---|---|
| Expose useful FinOps context through a small, read-only MCP interface. | Create a production-ready autonomous cost-management platform. |
| Enable a coding agent to propose or implement a repository change. | Allow an agent to modify production Azure resources directly. |
| Run a repeatable baseline-versus-candidate load test. | Guarantee invoice-level accuracy from a short-lived test. |
| Calculate an evidence-based estimated cost per workload unit. | Optimize every Azure service or workload pattern. |
| Publish an auditable pull-request summary. | Replace existing FinOps governance or approval processes. |

## 4. Baseline user journey

| Step | Experience | Evidence produced |
|---|---|---|
| 1 | Developer asks the coding agent to reduce cost while preserving repository-defined service-level objectives. | Optimization objective and constraints |
| 2 | Agent inspects source code, infrastructure-as-code, configuration, and repository guidance. | Candidate cost drivers |
| 3 | Agent calls MCP tools for FinOps, pricing, utilization, and recommendation context. | Grounded cost context |
| 4 | Agent proposes or implements one bounded change in a feature branch. | Diff, rationale, assumptions |
| 5 | GitHub Actions deploys isolated baseline and candidate environments and runs equivalent load profiles. | Performance and consumption measurements |
| 6 | Estimator calculates unit cost for both implementations. | Cost-per-workload-unit comparison |
| 7 | Workflow publishes a pull-request report with savings, performance impact, confidence, and tradeoffs. | Decision-ready review artifact |

## 5. Logical architecture

| Layer | Baseline components | Primary responsibility |
|---|---|---|
| Developer experience | GitHub repository; coding agent; repository instructions | Accept the optimization goal, inspect the solution, and manage code changes. |
| Cost intelligence | FinOps toolkit / FinOps hubs; Azure Cost Management exports; Azure pricing; Azure Advisor recommendations | Provide normalized cost, usage, pricing, and opportunity signals. |
| Agent integration | Read-only MCP server | Expose bounded tools and return structured, attributable context. |
| Experiment orchestration | GitHub Actions; Bicep; isolated Azure test resources | Deploy repeatable baseline and candidate configurations. |

| Layer | Baseline components | Primary responsibility |
|---|---|---|
| Validation | Azure Load Testing; Azure Monitor; Application Insights | Run equivalent workloads and collect latency, throughput, error, and resource metrics. |
| Analysis and reporting | Unit-cost estimator; pull-request report generator | Combine measured consumption and pricing; publish the decision record. |

## 6. Initial MCP tool contract

| Tool | Purpose | Hack priority |
|---|---|---|
| get_cost_summary | Return summarized historical cost for a bounded scope and time period. | Core |
| get_cost_by_resource | Identify material resource-level cost drivers. | Core |
| get_optimization_recommendations | Return Advisor or FinOps recommendation context. | Core |
| get_current_pricing | Retrieve pricing inputs needed by the estimator. | Core |
| get_resource_utilization | Return selected utilization signals when available. | Target |
| estimate_workload_cost | Calculate estimated cost for a defined workload unit. | Target |
| compare_experiment_results | Compare baseline and candidate measurements. | Target |
| record_optimization_experiment | Persist assumptions and experiment metadata. | Stretch |

## 7. Cost and performance measurement model

The hack should avoid claiming invoice-level cost accuracy. The preferred measure is estimated cost per workload unit, calculated from a controlled test and current price inputs.

| Measure | Examples |
|---|---|
| Workload unit | 1,000 API requests; one document processed; one completed workflow |

| Measure | Examples |
|---|---|
| Performance guardrails | P95 latency, throughput, error rate, availability during test |
| Consumption signals | Compute duration, instance count, memory, executions, storage transactions, network transfer, tokens where applicable |
| Comparison | Baseline unit cost versus candidate unit cost, plus percentage change |
| Confidence indicators | Test duration, sample size, pricing timestamp, excluded charges, extrapolation assumptions |

## 8. Security and governance baseline

- Use managed identity or workload identity federation where practical.
- Keep MCP access read-only for Azure cost, resource, and recommendation data.
- Limit all queries to an explicitly configured subscription, resource group, or FinOps scope.
- Do not place secrets, raw credentials, or sensitive billing exports in agent prompts or repository content.
- Require a human to review and merge code or infrastructure changes.
- Deploy experiments to isolated, budget-limited resources with automated teardown.
- Treat cost recommendations as advisory and report reliability, performance, and operational tradeoffs.

## 9. Open design decisions

| Decision | Baseline hypothesis | Validation needed |
|---|---|---|
| Source of normalized cost data | Use FinOps hubs or toolkit-aligned data where available. | Confirm hack sandbox access and data freshness. |
| Sample workload | Small API or event-processing workload deployed through Bicep. | Choose a scenario with measurable, safe optimization levers. |
| Agent host | GitHub Copilot coding agent or another MCP-enabled coding environment. | Confirm MCP support and automation boundaries. |
| Price calculation | Use current public or enterprise-appropriate price inputs plus test consumption. | Determine available price source and discount handling. |
| Experiment topology | Deploy equivalent baseline and candidate environments. | Balance accuracy against sandbox time and budget. |
| Optimization scenario | Rightsizing, scaling configuration, caching, logging volume, or service selection. | Select one change that produces a visible result. |

## 10. Success criteria

- The coding agent successfully retrieves grounded cost context through MCP.
- The agent produces one explainable optimization recommendation and a repository change or actionable patch.
- The workflow runs an equivalent load profile against baseline and candidate implementations.
- The report calculates a unit-cost comparison and includes performance guardrails.
- The result includes assumptions, limitations, and a traceable evidence trail.
- The sample can be packaged as a reusable reference pattern after the hack.

## 11. Expected evolution

This baseline is expected to change as the team learns which FinOps datasets are accessible, how quickly test telemetry becomes available, which coding-agent environment provides the best MCP experience, and which workload produces the clearest optimization story. Changes should be captured as short architecture decision records rather than silently replacing assumptions.